

The Weighted Directed Tree Augmentation Problem: Motivations, History, Uses, and Implementation

Oscar Guevara
Emily Navarro
Michael Dereje
Kobi Eng

MATH 482 COMB ALGORITHMS
4 - 24 - 26

AI Usage: AI was used in forming a boilerplate based on the pseudocode as well as generating test cases. Other than that, there was no AI usage in any of the writing portions of the project.

The Problem

The weighted directed tree augmentation problem (WDTAP) is a generalization of both the Tree Augmentation Problem (TAP) and the Directed Tree Augmentation problem (DTAP). In the WDTAP, we are given an oriented tree $T = (V, A)$, where the underlying undirected edges form a tree and we are also given a set of directed edges called links, each with a positive cost. It's important to note that the direction of the tree edges are arbitrary and do not need to point toward or away from a root. The goal is to select a minimum cost subset of links that protect the tree against the failure of any single tree edge. In order to formally describe this requirement, fundamental di-cuts must first be defined.

As an example, consider a directed tree edge $a = (u, v)$. If this edge is removed temporarily, the tree is split into two different connected components. One component contains u , while the other component contains v . Since the removed edge originally allowed movement from u to v , the augmentation must provide an alternate directed path that reconnects the two sides in the opposite direction. A directed link is said to cover the fundamental di-cut of edge a if it goes from the component containing v to the component containing u [1].

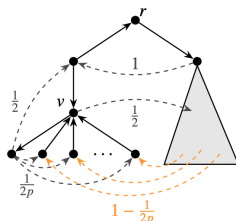
To reiterate, the objective of the WDTAP is to select a minimum cost set of links such that every tree edge has its corresponding fundamental di-cut covered. For example, suppose a tree contains a directed edge $(a \rightarrow b)$. Removing this edge separates the tree into two groups of vertices, one containing a and one containing b . In order to protect this cut, at least one directed link must cross from the b side back to the a side. This guarantees that connectivity can still be maintained even if the original edge fails[1].

The difficulty of the WDTAP comes from the fact that one link can cover multiple di-cuts simultaneously. Therefore, the algorithm has to balance two competing goals, selecting inexpensive links, while also maximizing how many cuts each chosen link covers. This interaction between costs and coverage is what makes this problem NP-hard[1].

Meike Neuwöhner, Olha Silina, and Michael Zlatin developed a $1.75+\epsilon$ approximation algorithm for the WDTAP, improving upon the previously best known factor of 2. The WDTAP is a minimization problem, meaning that it wants a result that is as small as possible. When it specifically comes to minimization problems, a $1.75+\epsilon$ approximation means that the algorithm always returns a solution that has a cost that is at most $1.75+\epsilon$ times the optimal solution.

Since the WDTAP is NP-hard, there is no known polynomial time algorithm that can always compute the exact optimal solution efficiently[1]. Because of this Approximation algorithms like this one have a tradeoff between efficiency and optimality. Even though the improvement from 2 to $1.75+\epsilon$ appears to be small, it is actually a pretty big leap and improvement from the worst case guarantee. A simple example is thinking in terms of numbers, suppose an algorithm has the optimal answer of 50,000. With the 2 approximations, it means that at the worst case scenario the algorithm would be 100,000. With the $1.75+\epsilon$ approximation it would be 87,500. It might not seem like a lot, but it's a substantial leap.

A key tool used in the paper is linear programming (LP) relaxation of the problem. In the original optimization problem, each link decision is integral. A link is either selected or not selected and it can be represented using binary variables, where $x_L = 1$ means the link is chosen and $x_L = 0$ means it wasn't chosen. With this being said the optimization problem can then be written as minimizing the total link cost while ensuring that every fundamental di-cut is covered, subject to the condition that every fundamental di-cut has at least one covering link. When it comes to solving this integer problem it is difficult computationally. The LP relaxation simplifies the problem by replacing binary constraints with fractional constraints, so instead of $x \in \{1, 0\}$, its $0 \leq x \leq 1$. This allows links to be chosen fractionally, making the problem solvable in polynomial time using linear programming methods[1].

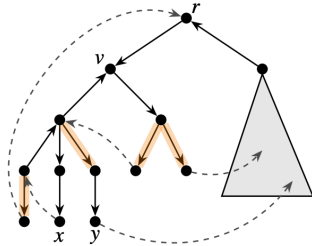


Source: Meike Neuwöhner, Olha Silina, Michael Zlatin, A Better-Than-2 Approximation for the Directed Tree Augmentation Problem, 2026

Although fractional solutions are not valid final answers, they help provide useful lower bounds on the optimal solution and reveal structural information about the instance. The paper uses the structure through concepts

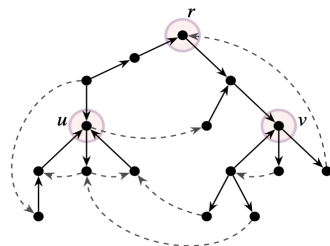
such as bounded visible width, willows, and link splitting in order to convert fractional solutions into near optimal integral solutions[1].

The concept of bounded visible width is used in the paper to control the complexity of how different parts of the tree interact through links. It limits how many independent di-cuts can be simultaneously influenced by any given region of the tree. Without this restriction, a single link can interact with many unrelated parts of the structure, making it more difficult to analyze how costs should be assigned. By ensuring that only a bounded number of cuts are active at any time within a subtree, the algorithm gains a controlled local structure. This bounded interaction is key because it prevents excessive overlapping previously stated to lead to the standard factor-2 approximation[1].



An example of visible width, at v is at least 4, shown by the four orange-shaded arcs in its subtree. This is an ancestor-free Set of down-arcs which are all visible to v . Source: MeikeNeuwohner, Olha Silina, Michael Zlatin, A Better-Than-2 Approximation for the Directed Tree Augmentation Problem, 2026

The other key tool presented in the paper are willows that refers to specifically decomposed subinstances of the tree where the underlying structure becomes incredibly simple and more organized. In willow instances, the tree is specific in the way that it behaves in a way that allows the linear programming relaxation of the problem to become tight, meaning that the fractional solution closely matches the integral solution. This makes willows incredibly useful as they represent regions of the instance that can be solved optimally rather than approximately. To ensure that links interact cleanly with these structured regions, the algorithm also uses link splitting where individual links are decomposed into smaller components that are assigned to specific parts of the decomposition. It prevents a single link from simultaneously affecting multiple regions in an uncontrolled way. Together, these techniques allow the instance to be broken into manageable pieces where local reasoning replaces global complexity, allowing there to be a tighter cost analysis and contributing to the improved $1.75+\epsilon$ approximation guarantee[1].



An example of a willow (choosing $W = \{r, u, v\}$). Vertex u is down-independent, vertex v is up-independent, and the root r is both. All links are either up-links, down-links, or have their apex in W . Source: MeikeNeuwohner, Olha Silina, Michael Zlatin, A Better-Than-2 Approximation for the Directed Tree Augmentation Problem, 2026

Motivation

As it will be said later, the origins of who first proposed this algorithm are unknown. Trees have become such a fundamental part of discrete mathematics that there is no singular source one can point to for their origin. The earliest known forms of trees came from general graph theory, with the first instance of such being a paper written in 1736 by Leonhard Euler titled *Solutio Problematis ad Geometriam Situs Pertinentis* about the Seven Bridges of Königsberg. This paper was the very first instance of graph theory being put into practice, and it details the usage of letters to denote the land masses a traveler can leave from or arrive at, now called vertices, and the bridges that connect these masses, now called edges. Though bridge traversal is a far cry from the graphs known today, this paper served as a foundation for the graph theory that would spawn from it.

Directed trees of any form can be used for a multitude of applications. One paper about the topology and structure of directed trees talks about how they can be used for “modeling hierarchical systems such as task schedules, syntax trees, inheritance relations, and decision flows” [2]. These are not the only applications of directed trees, of course, but they are the most common. By modeling these systems, this allows for a closer examination that

could prevent errors or conflicts further down the line. This is especially prevalent with inheritance relations and decision flows; modeling these systems in particular is incredibly important for computer scientists, as it allows them further control over their work and ensures they can make corrections where needed.

As stated previously, directed trees have multiple applications. These applications are not limited to discrete mathematics; the very origin of these graphs came from mapping bridge traversal, after all. One good example of more generalized application would come from genealogy, where directed trees are used to model family ancestry and relationships. In this case, the structure of the directed tree would be used to model the relationships between parent and child, dating back to a certain ancestor, with said ancestor acting as the root of the tree. Similarly, directed trees can be used to track the social network of a person, both in the real world and online. The person examined would serve as the root node, with their friendships and their closeness represented by parent-child relationships.

Directed trees are also used as the basis for the filing system used on computers. This is done by representing the nodes as files and directories and using the parent-child relationships helping to dictate which files are within which directories. This allows for efficient navigation of the file system; without it, users wouldn't be able to access the files on their computers so easily, nor would they be able to use the information within those files. [11] In a similar vein, this data structure makes it incredibly useful for database indexing for things like SQL databases. These trees make it possible to index and access this data without having to scan the entire database, thus making navigation and data storage both far more efficient.

Decision trees are named what they are for a reason; they get their name from the directed trees that they use as a basis. These decision trees are incredibly similar to decision flows; the only difference is how they are structured. In this case, they are used in machine learning in order to visualize and make decisions based on different features of the data involved. This approach makes it easier for even those not directly involved with the code to understand what is going on, and give their input on what decision should be made regarding the data. This is especially true when multiple factors are involved, where a change to any of them may drastically impact the final product. [3]

Directed trees can also be used in game development in several ways. There are of course decision trees that were just mentioned in order to keep track of what decisions the player may make and how to account for that. A similar structure may be used to decide what ending a player gets in their game, depending on the decisions that the player themselves made during gameplay. These directed trees can even be used for collision detecting in both 2D and 3D games, by using leaf nodes to represent regions of the virtual space and recursively dividing said space until a specific cell size is reached. Then checking for collisions becomes a matter of checking which region, or node in the tree, the player is in. [3]

More generally speaking, one might wonder why they would augment a tree, or why they would want to evolve the Tree Augmentation Problem (TAP) into Directed Tree Augmentation Problem (DTAP) or Weighted Tree Augmentation Problem (WTAP). The original motivation for TAP was for studying network connectivity, as will be stated later. The goal was to create a reliable network where the failure of one edge would not cause the entire tree to fail. These real-world applications are the basis for making TAP, and eventually evolving into DTAP and WTAP. Both of these were seen as more practical for real-world applications; network connections have a cost associated with them, and other networks like traffic flows and routing networks all come with inherent directions that TAP alone could not cover. This finally evolved into the Weighted Directed Tree Augmentation Problem (WDTAP) that this paper discusses, accounting for both the real-world costs and inherent directionality that DTAP or WTAP alone could not solve.

History

The Weighted Directed Tree Augmentation Problem is a recent evolution of different network augmentation problems, most notably the Tree Augmentation Problem (TAP), the Weighted Tree augmentation problem (WTAP) and the Directed Tree Augmentation Problem (DTAP). Although WDTAP was recently introduced, its foundations stem from classical questions in network reliability and survivability that were studied during the 1980s.

Seeing as the TAP is the origin of the WDTAP, in order to give an idea of how the problem evolved into different problems the problem will be formally defined. TAP in a way is the bare bones version of WDTAP that was defined in the problem section. The input of TAP consists of a tree $T = (V, E)$, and a set of edges called links, where each link connects two vertices in a tree. The idea is to once again minimize the amount of links necessary to cover the entire tree from being disconnected. In the weighted version WTAP, the idea is the same only now each link has a cost associated with it. The goal now is to select a minimum cost subset of links such that after adding the

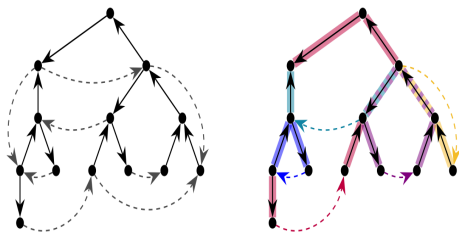
selected links to the tree, the resulting graph becomes 2-edge connected. Equivalently, every original tree edge must lie on a cycle formed using at least one selected link[1, 12].

TAP came about from the broader study of communication network reliability where researchers researched how to reinforce a network so that it remains connected even after the failure of a single edge. One of the foundational hardness results was given by Greg N. Frederickson and Joseph Ja'Ja who proved that Tap is NP-Hard[4]. Because of this complexity, TAP became an important problem in the study of approximation algorithms and combinatorial optimization.

As time passed, researchers developed increasingly better approximation algorithms for TAP. Early algorithms achieved approximation ratios of 2, meaning the solution cost was at most twice the optimal value, similar to the previous best approximation of WDTAP. Later work then improved the approximation substantially. As a matter of fact as of 2021 Tap now has a $1.393 + \epsilon$ approximation.

As time passed and constraints increased the problem continued to evolve in difficulty, the weighted tree augmentation problem appeared where instead of just being a simple problem where you are given a tree and links, this time each link had a weight associated with it. This increased the difficulty of the problem and increased the usability in research. The difficulty came about because now instead of just getting a minimum amount of links, you also had to minimize the total weight. There were similar structural properties to TAP because, the idea is fundamentally the same, make the tree 2 edge connected. Because this version of the problem was also NP-hard, it had a 2 approximation as well, and most recently now has a $1.5 + \epsilon$ as of 2022.

The directed version, DTAP, came about later when researchers expanded the restrictions and made it so edges have orientated constraints. This is an important evolution of the original problem because there are many real life systems like routing networks, communication systems, and traffic flows, that are inherently directional. Unlike in TAP, where connectivity is symmetric, DTAP requires preserving reachability across directed cuts, which adds to the complexity of the problem and forces the algorithm to change and evolve[1].



This is a DTAP instance on the left, with links drawn as dashed lines. A feasible solution is the graph on the right. The colors indicate the trees that are covered by each link. Source: MeikeNeuwohner, Olha Silina, Michael Zlatin, A Better-Than-2 Approximation for the Directed Tree Augmentation Problem, 2026

When it comes to the origins of trying to solve TAP and its close variants, it was usually conceptually simple, but computationally impractical. A brute-force approach to solving the problem is to enumerate all subsets of links and test whether each subset covers all required cuts. The downside to this is the fact that when the number of subsets grows exponentially, this brute force algorithm doesn't work most of the time except for times when instances are small. Greedy algorithms were also considered where the algorithm repeatedly selects the link with the best cost to coverage ratio. Although this seems to be a simple solution, if you think about it, the greedy method fails because it doesn't account for global interactions between links, like when a single link covers multiple tree edges or overlaps heavily with others. These early ideas were the ones that ended up showing what the real problem was. This problem being that link interactions are highly overlapping and local decisions can easily lead to global suboptimal solutions[1][6].

The main difficulty in TAP, WTAP, DTAP, and WDTAP is that links have global coverage effects. A single link may cover multiple tree edges or directed cuts, and multiple links may overlap on the same structures. This creates a complex dependency system where naive cost assignment leads to overcounting. The problem then gets even harder in DTAP because directionality removes the symmetry of the undirected case. A link that is useful in one direction can be useless in another, and ensuring correct coverage across all directed cuts introduces additional constraints. All these issues end up adding up and make all variants of the problem NP-HARD meaning there is no known polynomial time algorithm that can always compute the exact optimal solution[1][6].

So overall TAP originally began as a network reliability problem, evolved into directed and weighted variants as real world constraints were added, and ultimately led to WDTAP. The evolution from brute force and greedy method to LP based and structure driven algorithms reflects a shift from global unstructured reasoning to

carefully decomposed local analysis. This structural approach is what led to the modern structure guarantee of $1.75+\epsilon$, improving the long standing factor 2 barrier[1].

One Algorithm

The original algorithm that paved the way for the veritable field of improvements in network algorithm design was established in a paper written by Greg Frederickson and Joseph Ja'Ja'. Simply referred to as the Frederickson-JaJa algorithm, this was the first of its kind to provide a constant-factor approximation algorithm for an NP-Hard/NP-Complete problem like so. Formally, Frederickson and Ja'Ja' showed that graph augmentation problems, determining a minimum-cost set of edges to add to satisfy certain properties like strong connectivity or bridge-connectivity, are NP-complete even in the restricted case of the graph being initially connected, and presented approximation algorithms with constant worst-case performance ratios [4]. But these problems are not completely reflected in WDTAP due to how this problem doesn't simply ask for some existing path, but instead asks for the minimum weight set of edges. This takes problems like strong connectivity and bridge connectivity as a base, and pushes them even further, proving how this problem would remain NP-Hard even if the graph was complete. In making the graph complete, the running time would go from $O(E \log V)$ to $O(V^2 \log V)$, which is hardly an improvement. This is very unlike many other graph problems that have been solved by researchers approaching the problem from different angles in a similar manner. It would be easy to see how much time could have been wasted by many other researchers trying to cover something uncoverable, so this paper not only introduced a never before seen lower bound, but pointed future researchers away from a time consuming dead end. Modern optimizations have taken this algorithm to further heights, but the basis for these optimizations or different procedures entirely originated from this paper.

The reason this algorithm was so revolutionary was in part due to how it gave up on solving the problem directly and split it into simpler subproblems: finding both the reverse spanning arborescence (RSA) and spanning arborescence (SA), and then returning the union of both along with the given graph G . We begin by taking the graph G and making a copy G' . This is the graph that we will be modifying while leaving the original graph as is for bookkeeping purposes. Because we want to find the cheapest additional cost, we must set the existing edge weights to zero so that the chosen algorithm knows to prioritize the cheapest candidate links.

We would then run an algorithm like Edmonds' to find the RSA, commonly defined as a directed acyclic spanning subgraph. This subgraph treats one vertex s as the root, and all vertices except for s have exactly one leaving edge. This is to guarantee that there is a path from every vertex to the root. The next step would simply be a repeat but for the SA. An SA is also defined similarly to an RSA, but the difference would be that all vertices except for s have exactly one entering edge. In simpler terms, the directionality of all edges has been flipped or reversed.

This is to guarantee a path from the root to every vertex, and is also calculated by way of Edmonds' algorithm. By taking the union of the edges selected in both passes along with the original graph G , the algorithm would return a set of edges that guarantee a path from every vertex to the root, and from the root back to every vertex, while being mindful of the candidate links' weight. In other words, we can ensure that the graph is strongly connected, providing a solution that is within twice the cost of the optimal set of edges.

The running time of this algorithm simplifies to $O(E \log V)$. Flipping the edges in the beginning to calculate the arborescence has a negligible cost, as does the clean up—concerning the subtraction of the original edges present in G —at the end of the algorithm. Therefore, the main cost comes from running Edmonds algorithm each time, once for the upwards and once for the downwards spanning arborescence, and then drops any constants. $O(2 * E \log V)$ simplifies to $O(E \log V)$.

It is worth noting how this 2-approximation connects to the broader WDTAP result discussed in the beginning of this paper. The factor-2 bound arises because each of the two independent arborescence passes costs at most OPT, yielding a combined bound of $2 \times \text{OPT}$. The breakthrough of Neuwöhner, Silina, and Zlatin's $1.75+\epsilon$ result is breaking this barrier by exploiting the structured overlap of willows and visibly k -wide instances, by which large portions of the instance can be handled optimally rather than at twice the cost [1].

ALGORITHM

Input. $G(V, A)$ a complete directed graph with cost function $c: A \rightarrow \mathbb{Z}^+$, and $G_0 = (V, A')$ a subgraph of G .

Output. A set of edges $A'' \subset A - A'$ such that $(V, A' \cup A'')$ is strongly connected.

1. For each edge $\langle u, v \rangle$ in A , let $d\langle u, v \rangle = 0$ if $\langle v, u \rangle \in A'$ and let $d\langle u, v \rangle = c\langle v, u \rangle$,

otherwise.

2. Find a minimum weight spanning arborescence $T = (V, A'_a)$ with root $r \in V$ on (V, A) using d . Set $A_a = \{\langle u, v \rangle \mid \langle v, u \rangle \in A'_a\}$.
3. For each edge $\langle u, v \rangle$ in A , let $d'(\langle u, v \rangle) = c(\langle u, v \rangle)$. If $\langle u, v \rangle \in A' \cup A_a$, then set $d'(\langle u, v \rangle) = 0$. For all u , set $d'(\langle u, r \rangle) = \infty$.
4. Find a minimum weight spanning arborescence $T' = (V, A'_b)$ on (V, A) using d' .
5. Set $A'' = (A_a \cup A'_b) - A'$.

Implementation

Link to source code: https://drive.google.com/file/d/1_xGLybG-tW1jtPy0oDVRyfeGijjSwfkr/view?usp=sharing.

The implementation of the wdtap algorithm was not randomly generated nor drawn from a benchmark suite, but was constructed by AI to help form instances designed to go through different graph topologies. These small tests include a two-child tree rooted at node 0, a linear chain of four nodes, and a three-leaf star topology, rather than running large-scale experiments. The evaluation here is based on logical tracing of the structure of the algorithm, followed by the output of the algorithm from each test case.

The algorithm follows a two-pass minimum spanning arborescence (MSA) approach. In the first pass, it constructs a helper graph H containing the original tree edges at zero weight and all candidate links at their given costs. Then it computes a MSA on the reversed graph rooted at the given root, effectively finding a minimum cost branching that reaches all nodes when flipped. In the second pass, it builds a combined graph from the original tree, the flipped reverse spanning arborescence edges, and any remaining candidate links, blocks arcs entering the root with infinite weight to avoid trivial solutions, and runs another MSA. The result is the union of both passes' non-tree edges, which together are intended to provide strong connectivity augmentation at minimum cost. This follows closely to the Frederickson-Jaja algorithm, but also accompanies a directed approach, with the intuition behind the two-pass design is that the first pass identifies a cheap set of back-edges that create return paths toward the root, while the second pass fills in any remaining gaps in connectivity that the first pass may have missed.

Going through each of the cases gave us the outcomes we were looking for. For test 1 (two-child tree with nodes 0, 1, 2), the algorithm selects the link (1, 2) at cost 5, which makes sense. Adding a cross-edge between the two children combined with the existing tree edges is sufficient to create cycles back to the root through node 0. For Test 2 (linear chain $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$), the algorithm selects back-edges that allow traversal back toward the root. For Test 3 (star topology), it selects low-cost links back to the root from each leaf. In all three cases, the algorithm terminates quickly and produces a result, though the instances are tiny so their runtime is almost instantaneous and not really a good way to benchmark the performance of the algorithm. What is more telling is whether the outputs are actually optimal. For Test 1, selecting (1, 2) at cost 5 does appear to be the cheapest way to make the graph strongly connected, since the only alternatives involve buying back-edges directly to the root at cost 50 or 10, both of which are more expensive. For Tests 2 and 3, verifying optimality requires checking all feasible augmentation subsets, which is tractable by brute force at this scale but was not done as part of this evaluation.

As mentioned from the last paragraph, the three test instances are far too small to draw any meaningful conclusions about runtime performance. Each graph has at most four nodes and four candidate edges, meaning the algorithm completes in milliseconds. The computational complexity of the approach is dominated by the two calls to `minimum_spanning_arborescence`, which NetworkX implements using Edmonds' algorithm (also known as Chu–Liu/Edmonds' algorithm). This algorithm runs in $O(E \log V)$ time with efficient implementations, or $O(VE)$ in simpler versions, making it polynomial and well-suited to moderate-sized graphs. However, the overall augmentation problem being solved here is NP-hard in its most general form, and this heuristic makes no claim of polynomial-time optimality. For graphs with hundreds or thousands of nodes and a dense set of candidate links, the two MSA calls would still be fast, but the quality of the solution relative to the true optimum could degrade in ways that are invisible without benchmarking against exact solvers or known lower bounds.

Overall, the code is readable and neatly labeled. Using NetworkX for its built-in support with the minimum spanning arborescence function, rather than implementing Edmonds' algorithm from scratch, reduces the risk of low-level bugs. The program should be a good starting point for exploring the directed tree augmentation problem,

and classmates could use it as a reference implementation for small instances. However, it shouldn't be trusted blindly for correctness on larger or more complex inputs without more rigorous testing against known optimal solutions or established benchmarks. If the goal is to use this code as a baseline for comparison against one's own algorithm, it would be worth establishing a small suite of instances with known optimal solutions first, so that both algorithms can be evaluated on equal footing.

REFERENCES

- [1] Meike Neuwohner, Olha Silina, Michael Zlatin, “A Better-Than-2 Approximation for the Directed Tree Augmentation Problem”, SIAM, <https://doi.org/10.1137/1.9781611978971.56>, 2026
- [2] Enis Olgac, “Topology and Structure of Directed Trees: An Incremental Encoding”, <https://www.intechopen.com/chapters/1219247>, 2025
- [3] NC State. “File Tree and Pathnames – E 115: Introduction to Computing Environments.” *Ncsu.edu*, 2025, e115.engr.ncsu.edu/accessing-nfs/file-system-hierarchy/.
- [4] Frederickson, Greg N., and Joseph Ja’Ja. “Approximation Algorithms for Several Graph Augmentation Problems.” *SIAM Journal on Computing*, vol. 10, no. 2, May 1981, pp. 270–83. <https://doi.org/10.1137/0210019>
- [5] William Evans, Mohammad Ali Safari, “Directed One-Trees”, University of British Columbia, <https://www.cs.ubc.ca/~will/papers/dir1tree.pdf>, 2005
- [6] Zeev Nutov, “On the Tree Augmentation Problem”, <https://arxiv.org/pdf/1703.07247>, 2018
- [7] Peter Dankelmann, Wayne Goddard, Ortrud Oellerman, Henda C Swart, “Augmenting trees so that every three vertices lie on a cycle”, University of Natal, Durban, 2002
- [8] Walgo Galvez, Fabrizio Grandoni, Afrouz jabal Ameli, “On the Cycle Augmentation Problem: Hardness and Approximation”, 2021
- [9] Baeldung. “Real World Examples of Tree Structures | Baeldung on Computer Science.” *www.baeldung.com*, 28 Oct. 2021, www.baeldung.com/cs/tree-examples.
- [10] Narsingh Deo, “Graph Theory With Applications To Engineering And Computer Science”, Washington State University, [Graphtheory.pdf](#), 1976
- [11] Khuller, S., Thurimella, R. (1992). “Approximation algorithms for graph augmentation.” *ICALP 1992. Lecture Notes in Computer Science*, vol 623 https://doi.org/10.1007/3-540-55719-9_85
- [12] Jacob Thomas Restanio, “An Empirical Analysis of Approximation Algorithms for the Unweighted Tree Augmentation Problem”, West Virginia University, [Viewcontent.cgi](#), 2023

Credits

Oscar Guevara worked on the problem, problem history, and reference sections. Emily Navarro wrote the motivation section, and helped collect references to be used. Michael

Dereje researched the algorithm, and wrote the code for the implementation. Kobi worked on researching the algorithm and implementation.